PrivateCall — System Design and Implementation Report

Version 1.2 · October 2026

Prepared from the PrivateCall 1.2 source archive. The archive was rebuilt and reproduces the delivered file `PrivateCall.html` exactly (SHA-256 `6310489f…7a461268b`).

Abstract. PrivateCall is a one-to-one voice and video calling tool delivered as a single HTML file that people open directly from disk in a desktop browser. It has no server, account or installation. The two browsers negotiate a WebRTC connection through two text messages, an *invitation* and a *reply*, which the people carry between themselves with any messaging app. Media are encrypted with DTLS-SRTP, which the browser provides. PrivateCall adds a six-digit security code that both people compare to detect a party who substituted keys in the copied messages. The code is derived through a commit-then-reveal exchange modelled on ZRTP. By default, a person's microphone and camera are not attached to the outgoing streams until that person has confirmed the code. This report describes the architecture, call set-up, networking, the security and authentication design, the message formats, the application logic, privacy properties, build and testing, and the system's limits. It is based on the 1.2 source code, which takes precedence over earlier documentation where the two differ (Section 10.2). The report makes no claim that PrivateCall is anonymous, censorship-resistant, independently audited, or secure against sophisticated attackers.

1 Introduction

1.1 What the system is

PrivateCall is built on the browser's WebRTC interfaces [22]. A call needs two copies of the same file, `PrivateCall.html` (about 145 KB), opened in two browsers on two computers. One person chooses *Start a call*, which produces an invitation. The other chooses *Join a call*, pastes the invitation, and gets a reply to send back. When the first person pastes the reply, the browsers connect, each screen shows the same six-digit code if nobody has interfered, and the people compare it before talking freely. The scope is deliberately narrow: two participants, no group calls, no chat, no recording, no contact list, and no persistent storage.

1.2 Sources and conventions

The primary source for every statement is the 1.2 source code (`src/app/*.js`, `src/index.html`, `build.mjs`) and its test programs. Function names are given as `module.function` with the file in `src/app/` unless stated otherwise. Code listings are real excerpts, reproduced from the source with their original line numbers; nothing in a listing is simplified. Formulas in the text are mathematical descriptions of that code.

Because PrivateCall delegates most of the work to the browser, the report labels where behaviour comes from, as summarised in Table 1.

Table 1: Labels used to attribute behaviour and evidence.

App	Logic written in PrivateCall's own JavaScript (for example the security code, input checks, media gating).
Browser	Provided by the browser's WebRTC, WebCrypto or media stack (ICE, DTLS, SRTP, codecs, echo cancellation). PrivateCall only configures it.
External	Third-party services contacted over the network: public STUN servers and an optional TURN relay.
Tested	Observed in an automated test that was run (Section 9).
Reasoning	Follows from the design or from standards, but not demonstrated by a test.
Unverified	Not checked; needs other browsers, real networks or real hardware.

2 System architecture

Figure 1 shows the parts of the system and the three kinds of traffic: the signaling messages, which travel through people and channels outside PrivateCall; the peer-to-peer media and control traffic; and the network helper servers.

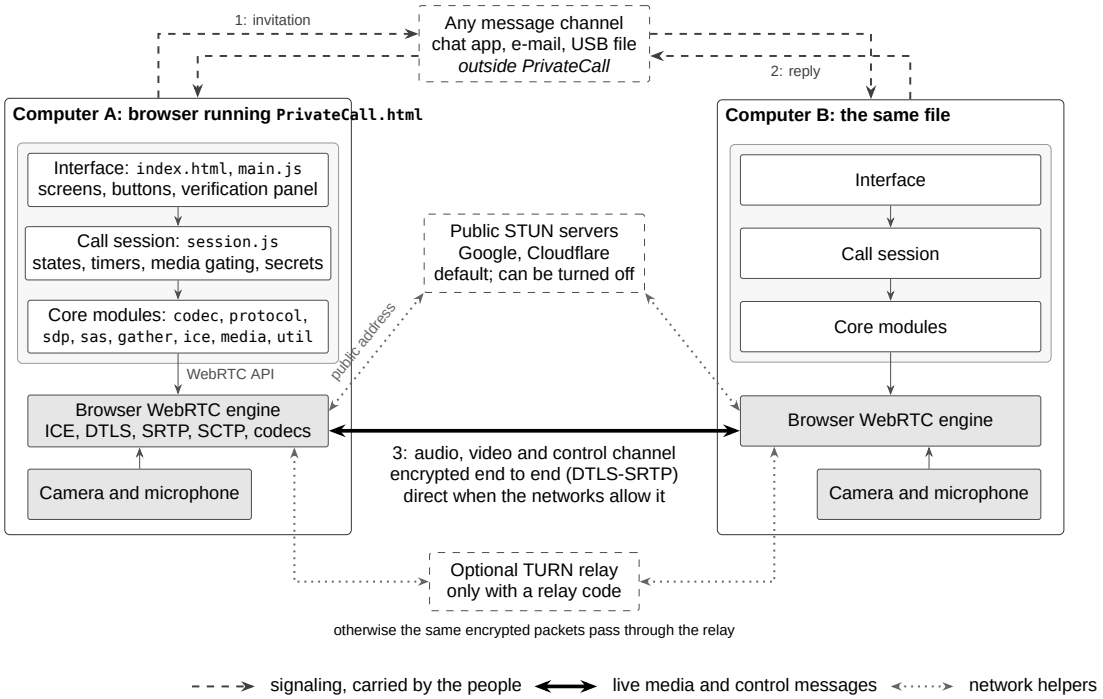


Figure 1: Architecture and communication paths. Only the two browsers hold encryption keys. The message channel, STUN servers and TURN relay are outside PrivateCall.

The application is split into eleven ES modules that the build concatenates in a fixed order (Table 2). The lower modules contain no user-interface code and are unit-tested directly under Node.js. `session.js` owns one call: the `RTCPeerConnection`, local media, all timers and all secret values. `main.js` drives the screens and translates session events into interface changes.

Table 2: Source modules in build order (`build.mjs`, `ORDER`).

Module	Lines	Responsibility
`util.js`	167	Strict UTF-8, canonical base64/base64url, `crypto.getRandomValues`, SHA-256 via WebCrypto, length-prefixed encoding, constant-time compare, bounded inflate.
`codec.js`	117	Invitation/reply text envelope: compression, header, checksum, base64 lines, tolerant decoding.
`sdp.js`	139	Defensive SDP inspection; fingerprint extraction; candidate splitting for re-supply.
`protocol.js`	86	Exact-key schemas for invitation, reply and data-channel control messages.
`sas.js`	53	Commitment and security-code derivation.
`ice.js`	68	ICE server configuration; relay-code parsing.
`gather.js`	52	Decision rule for when candidate gathering is finished.
`media.js`	211	Device access, error classification, codec preferences, sender parameters, level meter.
`session.js`	840	Call session: state, certificates, peer connection, gating, verification, statistics, teardown.
`i18n.js`	309	Interface strings (English only in 1.2) and right-to-left support.
`main.js`	1021	Screens, buttons, copy/paste/file handling, dialogs, call view.

Single-file build. `build.mjs` (131 lines, no dependencies) strips the `import/export` statements, rejects dynamic imports, `require`, duplicate top-level names and any literal `</script` or `</style`, wraps the modules in one strict-mode function, and inlines script and stylesheet into `index.html`. It then computes SHA-256 hashes of the inline script and style and writes a Content Security Policy that allows exactly those two

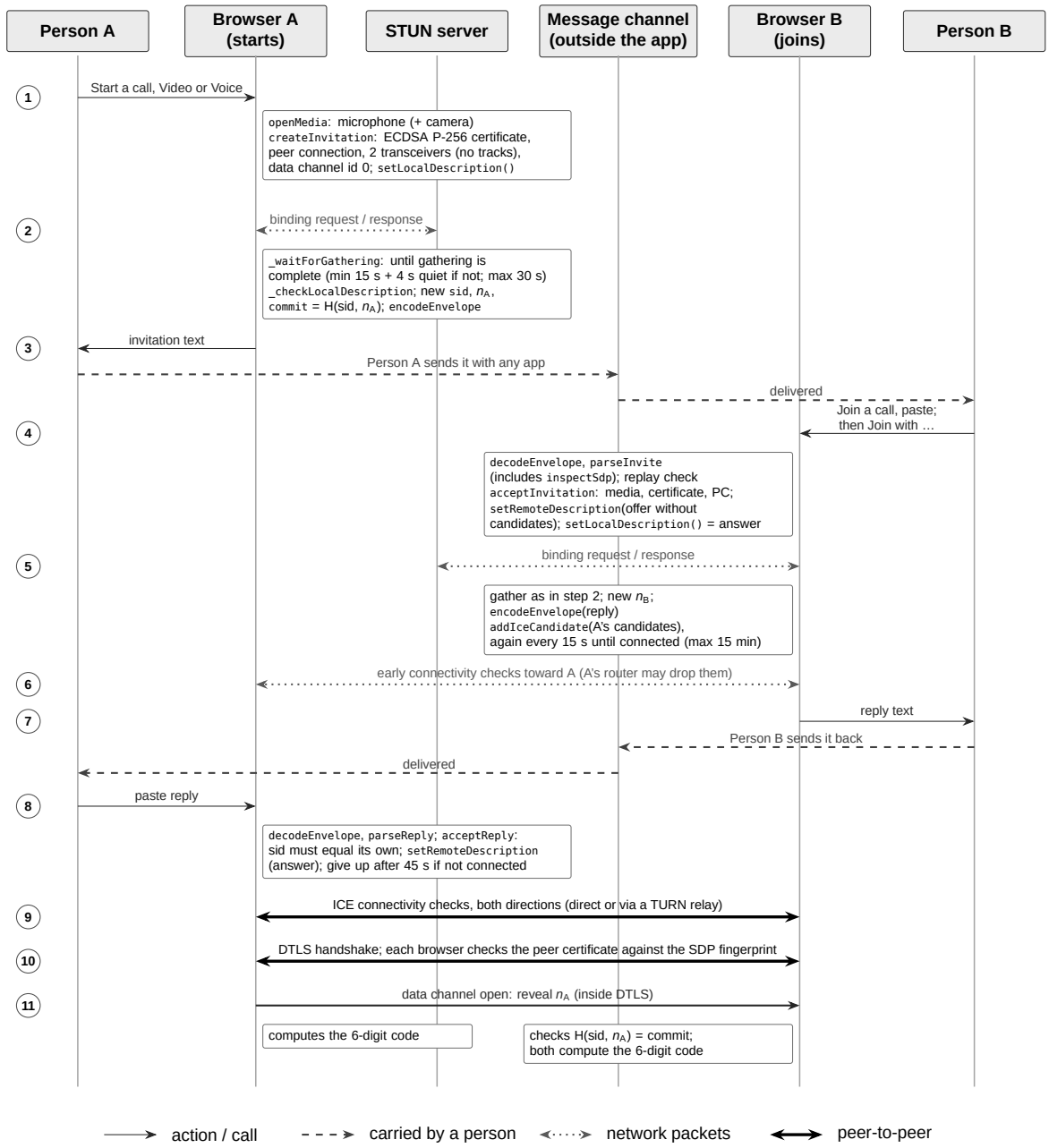


Figure 2: Call establishment, from *Start a call* to the security code. H denotes the labelled, length-prefixed SHA-256 constructions of Section 5.2. Dashed arrows are carried by people; the message channel is any app they choose.

blocks and nothing else (Section 8). The output and its checksum are written to `dist/`; `node build.mjs --check` verifies that `dist/PrivateCall.html` matches the sources byte for byte.

3 Call establishment

WebRTC needs the two browsers to exchange an SDP offer and answer (JSEP, RFC 9429 [1]) containing media descriptions, ICE credentials and candidates, and the DTLS certificate fingerprint (SDP, RFC 8866 [2]). PrivateCall carries the offer in the invitation and the answer in the reply. Because each message is sent exactly once, PrivateCall uses *non-trickle* ICE on the sending side: it waits for candidate gathering before encoding a message. Figure 2 shows the complete sequence.

3.1 Starting side

After microphone (and, for a video call, camera) permission is granted, `createInvitation` (Listing 1) creates a fresh ECDSA P-256 certificate with `RTCPeerConnection.generateCertificate`, a peer connection, one audio and one video transceiver in `sendrecv` mode *with no tracks attached*, and a pre-negotiated data channel (`negotiated: true, id: 0`) named `control` (RFC 8831 [14]). Codec preferences are applied, the offer is created with `setLocalDescription()`, and the browser starts gathering candidates. The local SDP is then inspected with the same function used for received SDP, and the browser-reported certificate fingerprint is compared with the one in the SDP (`_checkLocalDescription`). If no candidate at all was gathered, the person is told at this point ("no-network" or, in relay-only mode, "relay-unreachable") rather than after the invitation has been sent.

Listing 1: `session.js`, beginning of `CallSession.createInvitation`. The remaining lines (286–304) encode the invitation and set the state to `awaiting-reply`.

```
  async createInvitation(introText) {
    if (this.state !== 'idle') throw new Error('bad state');
    this.state = 'preparing';
    const pc = await this._createPeerConnection();
    // Both senders start empty; tracks are attached only when allowed (see header).
    this.audioTx = pc.addTransceiver('audio', { direction: 'sendrecv' });
    this.videoTx = pc.addTransceiver('video', { direction: 'sendrecv' });
    applyCodecPreferences(this.audioTx, 'audio');
    applyCodecPreferences(this.videoTx, 'video');
    this._attachRemoteStreams();
    await pc.setLocalDescription();
    await this._waitForGathering();
    if (this._ended) return null;
    const info = this._checkLocalDescription('offer');

    this.sid = randomBytes(SIZES.SID_BYTES);
    this.nonceA = randomBytes(SIZES.NONCE_BYTES);
    this.commit = await makeCommit(this.sid, this.nonceA);
```

3.2 When gathering is finished

A candidate gathered after the message is produced is lost for the call, so the rule in `gather.gatherDecision` (Listing 2) favours completeness. The session polls it every 250 ms. With reachable servers the browser reports gathering complete quickly; the minimum and quiet periods only matter when a configured server never answers. In that case the invitation is produced after 15–30 s, and the interface explains the wait after 4 s.

Listing 2: `gather.js`, `gatherDecision`. `GATHER_POLICY` sets `minMs` = 15 000, `quietMs` = 4 000 and `maxMs` = 30 000.

```
export function gatherDecision(state, policy = GATHER_POLICY) {
  if (state.complete) return { done: true, reason: 'complete' };
  if (state.elapsedMs >= policy.maxMs) return { done: true, reason: 'cap' };
  if (state.elapsedMs < policy.minMs) return { done: false, reason: 'min' };
  if (state.msSinceLastCandidate < policy.quietMs) return { done: false, reason: 'active' };
  // A relay is the fallback that makes restrictive networks work. Until we hold a
  // relay candidate, keep waiting for any relay URL that has not answered yet.
  // Once we hold one, other relay transports (UDP/TCP/TLS to the same relay) add
  // no new reachability: all relay candidates are the relay's public address.
  if (state.relayUrls.length > 0 && state.attribution && !state.haveRelayCandidate) {
    const pending = state.relayUrls.filter((u) => !state.settledRelayUrls.has(u));
    if (pending.length > 0) return { done: false, reason: 'relay-pending' };
  }
  return { done: true, reason: 'quiet' };
}
```

The relay rule depends on the browser reporting which server produced a candidate or an error (the candidate's `url` attribute, or the error event's `url`). When that attribution is not available, the rule falls back to the quiet-period test.

3.3 Joining side

`main.handleInviteText` decodes the text and validates it (Section 6), then shows the call type and the invitation's age. `acceptInvitation` refuses an invitation already used in this window, creates its own certificate and peer connection, and applies the offer *without* its candidates (`sdp.splitCandidates`). It then answers, gathers under the same policy, draws the 32-byte value n_B, and encodes the reply.

The removed candidates are then supplied separately with `addIceCandidate`, the standard trickle interface (RFC 8838 [4]), and supplied again every 15 s with a modified foundation (`sdp.refreshFoundation`) until the connection is established or 15 minutes have passed. Each round makes the browser start new connectivity checks toward the same addresses. This exists because of a measured failure: in the network lab, port-restricted NAT routers dropped the joining side's early checks, and the router mappings they created expired before the reply arrived, so calls failed when the reply was added more than about 40 s after it was created. With re-supply, lab calls connected with the reply added after 90 s and 200 s (*Tested* in the lab only). Re-supplying reveals no new addresses and does not involve the certificate fingerprints. After 15 minutes the joining side shows a notice suggesting a new invitation; the session itself is not ended.

3.4 Applying the reply

`acceptReply` accepts a reply only while the starting side is in state `awaiting-reply` and only if its session identifier equals its own (constant-time comparison). It applies the answer and starts a 45 s timer; if no connection exists by then, the call ends as "Couldn't connect" with a list of the candidate types that were found. ICE connectivity checks (RFC 8445 [3]) and the DTLS handshake then run inside the browser Browser. The starting side has no time limit while it waits for a reply; the invitation stays usable as long as that window is open.

4 Networking and media

4.1 Connectivity configuration

`ice.buildIceConfig` builds the peer-connection configuration from three in-memory settings: use of public STUN servers (on by default), an optional relay, and relay-only mode. All media sections share one transport (`bundlePolicy: 'max-bundle'`), RTCP is multiplexed (`rtcpMuxPolicy: 'require'`), and no candidates are pre-gathered (`iceCandidatePoolSize: 0`).

The two default STUN servers External (RFC 8489 [5]) are `stun.l.google.com` (port 19302), operated by Google, and `stun.cloudflare.com` (port 3478), operated by Cloudflare. They tell each browser its public address and port and therefore learn that the address is setting up a call and when; they carry no call content. Their reachability from any particular country or network was not tested (*Unverified*).

When both sides are behind address-dependent ("symmetric") NATs (RFC 4787 [7]) or UDP is blocked, a direct path is impossible and a TURN relay External (RFC 8656 [6]) is needed. PrivateCall contains no relay address or credential. A relay operator generates a time-limited *relay code* with `tools/make-relay-code.mjs`: the code is `PCRELAY1:` followed by base64-encoded JSON holding 1–4 `turn:/turns:` URLs, a username of the form `<expiry>:<label>`, an HMAC-SHA1 credential derived from the operator's shared secret (the scheme used by coturn's `use-auth-secret` option [28]), and the expiry time. `ice.parseRelayCode` accepts only these keys, at most 2000 characters, and refuses expired codes. The secret itself never enters the code, but the code is a bearer credential until it expires. Only one participant needs a relay. Figure 3 contrasts the two kinds of path.

Relay-only mode sets `iceTransportPolicy: 'relay'` and stops contacting public STUN servers, so the invitation contains only relay candidates and the other person learns only the relay's address (*Tested* in the lab with TURN over UDP).

4.2 Media

Devices are opened by `media.openLocalMedia` with echo cancellation, noise suppression and automatic gain control requested, mono audio, and video at an ideal 1280×720 and at most 30 frames per second. If the camera cannot be opened for any reason other than a refused permission, PrivateCall retries with the

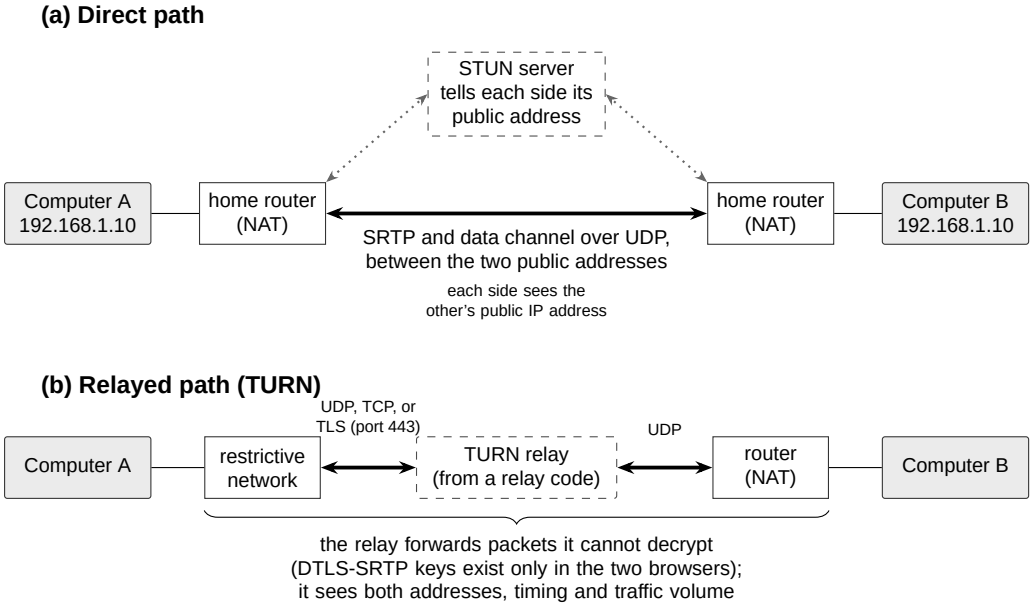


Figure 3: Direct and relayed media paths. Addresses are illustrative; both homes may use the same private range, as in the network lab.

microphone alone and continues as a voice call with a notice. Codec preferences are set where the browser supports `setCodecPreferences`: Opus with RED for audio; VP8 first, then VP9 profile 0, H.264 constrained baseline (`42e01f`, packetization mode 1) and RTX for video. These are codecs that WebRTC browsers are expected to implement [18, 19]. Once connected, `tuneSenders` asks for high priority on audio and limits video to 1.5 Mbit/s with low priority; this is best effort, and its effect was not measured. Received audio and video are attached to two separate `MediaStream` objects, so the browser does not synchronise lip movement with sound.

Encoding, packetisation, jitter buffering, congestion control and echo cancellation are browser functions Browser. In a local test call (same machine, synthetic devices) the receiver's statistics showed Opus in use, a jitter-buffer delay of about 32 ms against a 22 ms target, and 30 ms audio playout delay; this indicates that PrivateCall's configuration adds no buffering of its own. Latency on real networks was not measured (*Unverified*).

4.3 During the call

Every 2 s `_sampleStats` reads `getStats()` [23] and derives the route (direct or relay), transport protocol, round-trip time, audio packet loss over the interval, audio buffering delay, codec, video resolution and frame rate, and the SRTP cipher and DTLS version. These appear under *Details*. A quality label is shown as "good" when RTT < 300 ms and loss $< 2\%$, "fair" when RTT < 600 ms and loss $< 8\%$, and "poor" otherwise. If the connection state becomes `disconnected`, the person sees "Connection unstable", and the call ends as "Connection lost" unless it recovers within 30 s; a `failed` state after the first connection ends the call at once. There is no automatic reconnection with new messages.

5 Security and authentication

5.1 Threat model

The design assumes that the message channel may be read and modified by an attacker, that the network may be observed and manipulated, and that both computers and both copies of the HTML file are trustworthy. It builds on the WebRTC security analysis in RFC 8826 [16]. Table 3 summarises what each mechanism addresses.

Table 3: Threats and the mechanisms that address them.

Threat	Mechanism and status
Passive network observer	DTLS-SRTP encryption of media and the data channel [9, 10] Browser. Observed in Chromium 153: DTLS 1.3 [11], `SRTP_AES128_CM_HMAC_SHA1_80` (*Tested*, reported by `getStats`).
Key substitution in the copied messages (person in the middle)	Six-digit code over both fingerprints and both nonces, with a commitment App; media gated until confirmed App (*Tested* against a scripted attacker).
Attacker searching for certificates that make both codes match	Commit-then-reveal: n_{A} hidden until both DTLS sessions are fixed App (*Reasoning*, Section 5.3).
Attacker imitating a voice reading the code	Not prevented. Interface recommends comparing the code as text in a different trusted app App (*Reasoning*; not tested).
Malformed or hostile input	Size limits, exact schemas, strict SDP rules, bounded decompression App (*Tested*, including fuzzing).
Impersonation (attacker is the other endpoint)	Not detected by the code. People must recognise each other. (*Tested* as a documented limit.)
Compromised computer, malicious copy of the file, recording by the other person	Out of scope.

What the browser guarantees. During the DTLS handshake the browser checks that the peer's certificate matches the `a=fingerprint` line in the SDP it was given and refuses the connection otherwise (RFC 5763 [8], RFC 8122 [12], RFC 8827 [15]). DTLS-SRTP (RFC 5764 [9]) derives the SRTP keys from the handshake, so the keys exist only in the two browsers. The DTLS key exchange is ephemeral, which gives forward secrecy Browser. This protects the call only if each browser received the *genuine* fingerprint of the other. An attacker who controls the message channel can replace both fingerprints with its own and relay the call. Encryption alone cannot detect this; the security code exists to detect it.

5.2 The security-code protocol

Let $\mathrm{LP}(x_1, \ldots, x_k)$ be the concatenation of the fields, each preceded by its length as a 4-byte big-endian integer (`util.lengthPrefixed`), so that field boundaries cannot shift. Fingerprints enter as the text `"sha-256 AB:CD:…"` (algorithm in lower case, value in upper case). The protocol, shown in Figure 4, is:

1. The starting side A draws a 16-byte session identifier *sid* and a 32-byte value n_{A}, and puts $\mathit{commit} = \text{SHA-256}(\mathrm{LP}(\text{“PrivateCall commit v1”}, \mathit{sid}, n_{\mathrm{A}}))$ in the invitation next to its offer, which carries A's fingerprint fp_{O}.
2. The joining side B returns a 32-byte value n_{B} with its answer, which carries B's fingerprint fp_{A}.
3. When the data channel opens (the DTLS handshake is complete, so both certificates are fixed), A sends n_{A} in a `reveal` message. B checks it against *commit* and ends the call with "Security check failed" if it does not match or does not arrive within 20 s.
4. Both compute the code (Listing 3):

$$\mathit{code} = \left(\sum_{i=0}^{7} h_i \, 2^{8(7-i)} \right) \bmod 10^6, \qquad h = \text{SHA-256}(\mathrm{LP}(\text{“PrivateCall SAS v1”}, \mathit{sid}, \mathit{fp}_{\mathrm{O}}, \mathit{fp}_{\mathrm{A}}, n_{\mathrm{A}}, n_{\mathrm{B}})),$$

shown as six digits with leading zeros. Reducing 64 bits modulo 10^6 introduces a bias below 2^{-44}.

Listing 3: `sas.js`, commitment and code derivation (complete apart from the header comment).

```
const COMMIT_LABEL = 'PrivateCall commit v1';
const SAS_LABEL = 'PrivateCall SAS v1';
export const SAS_DIGITS = 6;

export async function makeCommit(sid, nonceA) {
  return sha256(lengthPrefixed([COMMIT_LABEL, sid, nonceA]));
}

export async function checkCommit(sid, nonceA, commit) {
  return bytesEqual(await makeCommit(sid, nonceA), commit);
}

export async function deriveSas({ sid, offerFingerprint, answerFingerprint, nonceA, nonceB }) {
  const h = await sha256(
    lengthPrefixed([
      SAS_LABEL,
      sid,
      fingerprintString(offerFingerprint),
      fingerprintString(answerFingerprint),
      nonceA,
      nonceB,
    ]),
  );
  // 64 bits reduced mod 10^6: bias is below 2^-44, negligible.
  let n = 0n;
  for (let i = 0; i < 8; i++) n = (n << 8n) | BigInt(h[i]);
  return (n % 10n ** BigInt(SAS_DIGITS)).toString().padStart(SAS_DIGITS, '0');
}
```

On the receiving side, `_onControl` accepts a `reveal` only on the joining side, only once, and only before a code exists (Listing 4). The commitment check uses a constant-time byte comparison.

Listing 4: `session.js`, the reveal branch of `_onControl`. On the starting side, `_onChannelOpen` (lines 504–517) sends `reveal` and computes the code; on the joining side it starts the 20 s timer.

```
  async _onControl(data) {
    if (this._ended) return;
    const m = parseControl(data);
    if (!m) return;
    switch (m.t) {
      case 'reveal':
        if (this.role !== 'answerer' || this.nonceA || this.sas) return;
        if (!(await checkCommit(this.sid, m.na, this.commit))) {
          this.end('security-failed', { cause: 'commit' });
          return;
        }
        this.nonceA = m.na;
        this._clear(this._revealTimer);
        await this._computeSas();
        break;
```

5.3 Why the commitment matters

Without the commitment, an attacker in the middle runs two separate DTLS sessions, one with each person, and knows every input to both codes before it must fix its own certificates. It could then generate certificates until both codes agree, about 10^6 attempts for a six-digit code. With the commitment, the attacker must send B a commitment before learning n_{A}. If it forwards A's commitment, it learns n_{A} only after both of its DTLS sessions, and hence the fingerprints in both codes, are fixed; if it substitutes its own commitment, it is bound to its own value. In either case the two codes coincide only by chance, with probability about 10^{-6} per attempt, and each attempt is a call that the people see and must accept. This argument mirrors ZRTP's hash commitment and short authentication string (RFC 6189 [17], Sections 4.4.1.1 and 7). It is *reasoning*: it has not been formally verified or independently reviewed. The tests exercise concrete strategies: substitution with a forwarded commitment, a forged commitment, a replaced or replayed

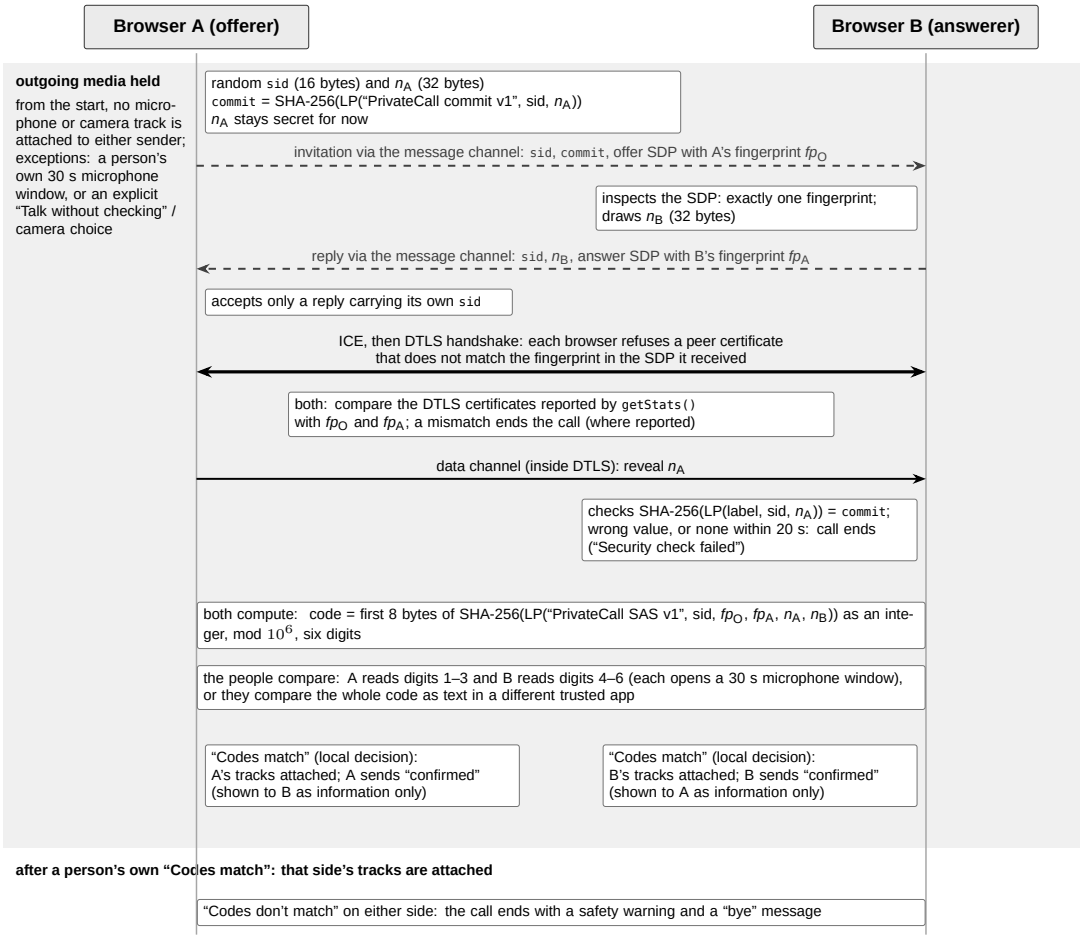


Figure 4: Commitment, reveal and verification. The shaded band marks the period during which neither side attaches its microphone or camera track to the outgoing streams, apart from the listed exceptions.

reveal, and a withheld reveal (Section 9).

5.4 The human comparison

The bound above assumes that the attacker cannot change what the people say or see during the comparison. The panel splits the code: the starting side is asked to say the first three digits and to hear the last three, the joining side the reverse, so speech in both directions is checked. For example, with the code `482 917`, the person who started the call says "four eight two" and should hear "nine one seven". A person clicks *Codes match* only when both halves agree with their own screen.

A spoken comparison has a known weakness: an attacker who relays the call can try to imitate or splice a voice to read out the code the victim expects. Such attacks have been demonstrated against cryptophone code comparison [27]. PrivateCall does not attempt to detect this. The panel therefore recommends comparing the code as text through a different app the people trust, and offers a *Copy code* button. The strength of that alternative depends on the second channel and was not tested. Whether people actually compare codes, and use the text option, was also not studied (*Unverified*).

5.5 Media gating before verification

Encryption protects the call against outsiders, but before verification the "other side" could be an attacker in the middle. PrivateCall therefore sends no microphone or camera media until the local person confirms the code. The rule is enforced on the RTP senders, not by muting: a sender that is not allowed has no track at all (`replaceTrack(null)`), so no audio or video from the device is encoded. Listing 5 and Figure 5 show the conditions.

Listing 5: `session.js`, the gating conditions and the audio sender update.

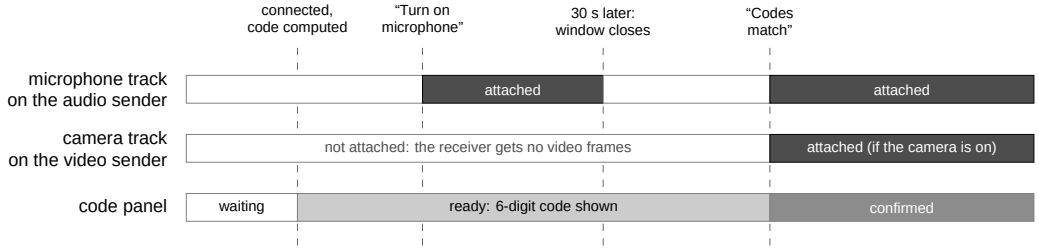
```
  // ---------- local controls ----------
  get videoAllowed() {
    return this.verify === 'confirmed' || this.allowUnverifiedVideo;
  }

  get audioAllowed() {
    return this.verify === 'confirmed' || this.allowUnverifiedAudio || this.speaking;
  }

  get sendingAudio() {
    return !!(this.audioTx && this.audioTx.sender.track && this.micOn);
  }

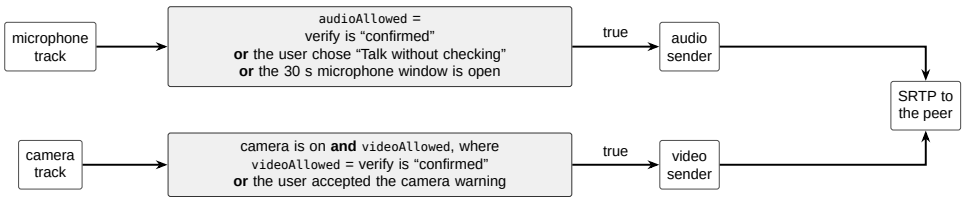
  async _syncAudioSender() {
    if (!this.audioTx || this._ended) return;
    const want = this.audioAllowed ? this.audioTrack : null;
    if (this.audioTx.sender.track !== want) {
      try {
        await this.audioTx.sender.replaceTrack(want);
      } catch (e) {
        console.warn('replaceTrack failed', e);
      }
    }
    this._sendState();
    this.emit('local-media');
  }
```

(a) One side's outgoing media during a call (example; not to scale)



The other side follows the same rule independently: its tracks are attached only after its own person clicks "Codes match" (or during its own microphone window). A "confirmed" message received from the other side changes nothing locally.

(b) The two conditions evaluated before every `replaceTrack()` call (`session.js`)



When a condition is false the sender's track is replaced by `null`: the transceiver stays negotiated but no audio or video from the device is encoded or sent. The "Microphone" button separately sets `track.enabled` (muted audio is sent as silence by the browser); turning the camera off stops the camera track.

Figure 5: Outgoing media for one side. (a) Example timeline. (b) The conditions; video additionally requires the camera to be on.

There are three ways to send before confirming, each initiated by the local person: *Turn on microphone* in the code panel opens the microphone for 30 s so the person can read the code; *Talk without checking* attaches the microphone after a warning dialog; and turning the camera on before confirmation shows a warning dialog whose "Share camera now" choice allows video. Confirmation is strictly local. A `confirmed` message from the other side is displayed as information only and never attaches local media. Clicking *Codes don't match* ends the call with a safety warning and sends `bye`.

5.6 Additional consistency checks

Several checks make sure that the fingerprint in the code is the one the browser enforces:

- `sdp.inspectSdp` requires *exactly one* distinct fingerprint across the whole SDP, with a SHA-256, SHA-384 or SHA-512 algorithm and the matching number of bytes, and a single `a=setup` value [13] (`actpass` in offers, `active` or `passive` in answers). SDP with two different fingerprints is rejected, so the code cannot be computed over one fingerprint while the browser uses another.
- Before sending its own message, each side compares the fingerprint in its SDP with the fingerprint the browser reports for its certificate (`getFingerprints`).
- After connecting, `_checkCertificates` reads the local and remote certificates that the DTLS transport actually used from `getStats()` and compares them with the fingerprints in the code. A mismatch ends the call. If the browser does not report them, the result is shown as "unavailable" in *Details*. This duplicates a check the browser already performs; it is defence in depth.

5.7 What the code does not prove

A matching code shows that no one replaced the keys between these two browsers for this call. It does not identify the other person: an attacker who answers the invitation itself obtains a matching code (a test confirms this). People must recognise each other. The code also offers no protection if either computer is compromised, if the HTML file was tampered with before use, or if the other person records the call.

6 Signaling format and input handling

6.1 The envelope

Invitations and replies must survive chat apps that re-wrap lines, add quote markers, insert invisible characters or apply formatting. `codec.encodeEnvelope` serialises the payload as JSON, compresses it with raw DEFLATE (RFC 1951 [20]) through `CompressionStream('deflate-raw')`, adds a five-byte header and an eight-byte checksum, and encodes the result in standard base64 (RFC 4648 [21]) split into 64-character lines between a marker and `:end` (Figure 6). In the automated tests a complete invitation, including its introductory sentence, was about 1,870 characters long.

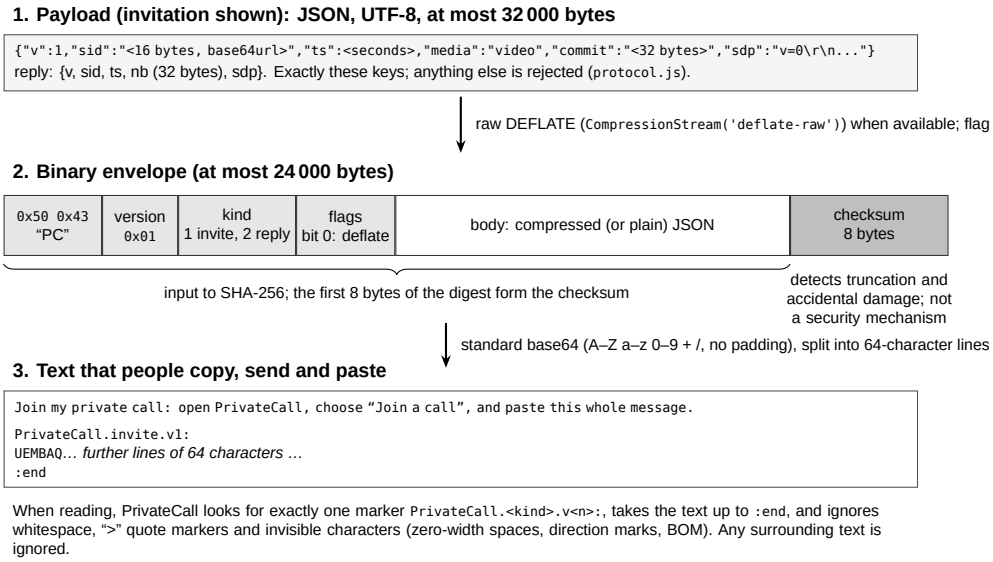


Figure 6: Envelope layout, from payload to the text that people copy.

The checksum is the first eight bytes of SHA-256 over header and body. It detects truncation and accidental damage only; anyone can recompute it. Protection against deliberate changes comes from the security code.

6.2 Validation

Decoding is strict and bounded. Table 4 lists the limits; every failure maps to a specific message in the interface (for example "incomplete", "corrupt", "version-newer", or "this looks like a relay code").

Table 4: Input limits and checks (`codec.js`, `protocol.js`, `sdp.js`, `main.js`).

Input	Rule
Pasted text	At most 40,000 characters; exactly one marker `PrivateCall.<invite|reply>.v<n>:` followed by `:end`.
Opened or dropped file	At most 64 KiB, read as text.
Decoded envelope	At most 24,000 bytes; magic `PC`, version 1, kind matching the marker, only flag bit 0, valid checksum.
Decompressed JSON	At most 32,000 bytes; decompression stops at the limit, which defeats decompression bombs.
Invitation payload	Exactly `v, sid, ts, media, commit, sdp`; `sid` 16 bytes, `commit` 32 bytes (base64url, canonical); `media` `video` or `audio`; `ts` a plausible Unix time.
Reply payload	Exactly `v, sid, ts, nb, sdp`; `nb` 32 bytes.
SDP	20–20,000 printable ASCII characters with CRLF line ends; starts with `v=0`; one audio, one video and one application section; one fingerprint; one setup value; consistent ICE username fragment (4–256) and password (22–256 characters).
Control message	At most 512 characters of JSON; exactly one of `reveal{na}`, `state{mic,cam}`, `confirmed`, `bye`, with exact keys and types.

Received text is never interpreted as markup. The interface writes it only through `textContent` or form values, and the Content Security Policy enforces Trusted Types with no policies, so HTML injection sinks such as `innerHTML` cannot be used. The source contains no `innerHTML`, `eval` or `new Function`.

Replay and freshness. The joining side remembers, in memory for the lifetime of the window, the session identifiers of invitations it has used, and refuses to join one twice. The starting side accepts only one reply, and only for its own identifier. A replayed invitation opened in a different window is not detected, but it cannot complete a call: the DTLS handshake requires the private key of the certificate in the invitation, which exists only in the original window (*Reasoning*). Timestamps are not an expiry mechanism. An invitation older than one hour produces a warning, and one dated more than ten minutes in the future produces a clock warning.

7 Application logic and state

`CallSession` has a session state and, once connected, a verification state (Figure 7). Every timer is registered with the session so that `end()` can clear all of them. Table 5 lists the timing constants.

Table 5: Timing constants (`session.js` `TIMEOUTS`; `gather.js` `GATHER_POLICY`).

Constant	Value	Effect
`minMs` / `quietMs` / `maxMs`	15 / 4 / 30 s	Gathering policy (Listing 2).
`gatherSlowNotice`	4 s	"Checking which network routes work..."
`candidateRefresh` / `Max`	15 s / 15 min	Joining side re-supplies candidates until connected.
`connect`	45 s	Starting side gives up after adding the reply.
`reveal`	20 s	Joining side waits for n_{A} after the channel opens.
`speakWindow`	30 s	Microphone window for reading the code.
`disconnected`	30 s	Recovery time before "Connection lost".
`statsInterval`	2 s	Statistics sampling.

Ending a call. `CallSession.end(reason)` runs once. For `local-hangup`, `code-mismatch`, `page-closed` and `security-failed` it first sends `bye`. It then clears all timers, closes the data channel and peer connection, stops every local and remote track (releasing camera and microphone), overwrites the byte arrays holding n_{A}, n_{B} and *sid* with zeros, drops the commitment, code and certificate references, and reports the reason to

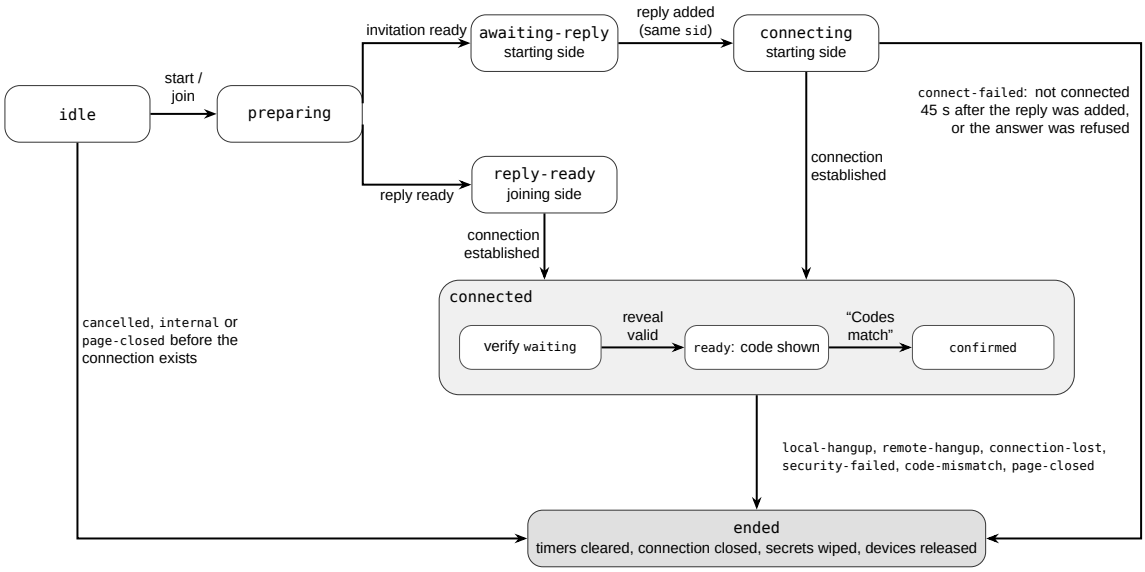


Figure 7: Session and verification states. The joining side moves from `reply-ready` directly to `connected`; only the starting side passes through `connecting`.

the interface. The overwrite is best effort: JavaScript may keep other copies (for example the base64 text of the messages) until garbage collection. The interface maps reasons to screens; for instance `code-mismatch` shows "Call ended for your safety" and `connect-failed` shows "Couldn't connect" with diagnostics. Closing the window during a call triggers the browser's leave-page prompt, and `pagehide` ends the session. If the other side's data channel closes during a call, the call ends as `remote-hangup`.

Interface. `main.js` implements a fixed set of screens: welcome, start, join, call and ended, plus dialogs for settings, privacy information, details and the two pre-verification warnings. Each screen states the next action. Messages can be copied (with a fallback when clipboard permission is unavailable), saved as `PrivateCall-invitation.txt` or `PrivateCall-reply.txt`, pasted, opened from a file, or dropped onto the window. In a call, M toggles the microphone and V the camera. The interface strings are English only in 1.2; the layout supports right-to-left languages, and digits can be rendered in a locale's own numbering system, but no Persian translation is included.

8 Privacy and local execution

PrivateCall runs entirely in the browser from a `file://` URL. Browsers treat such pages as secure contexts [26], which WebRTC and device access require. The page declares this Content Security Policy (built in `build.mjs`; hashes abbreviated):

```
default-src 'none'; script-src 'sha256-…'; style-src 'sha256-…'; img-src data:; media-src
'none'; connect-src 'none'; font-src 'none'; object-src 'none'; frame-src 'none'; worker-src
'none'; manifest-src 'none'; base-uri 'none'; form-action 'none'; require-trusted-types-for
'script'; trusted-types 'none'
```

The policy [24, 25] prevents the page from loading or fetching anything (`connect-src 'none'`) and from running any script other than the hashed one. It does *not* govern WebRTC traffic, which is the intended network activity. The referrer policy is `no-referrer`. Each end-to-end test also failed on any HTTP request or CSP violation; none occurred (*Tested* in Chromium).

Nothing is stored by the application: no `localStorage`, cookies or IndexedDB are used, settings and relay codes are held in memory only, and the invitation and reply text is cleared from the page when a call ends. Copies made by the people are outside PrivateCall's control: the clipboard, saved `.txt` files, and the messaging app's history keep the messages, including the network addresses in them. Table 6 lists what other parties can learn.

PrivateCall does not hide that a call takes place and makes no anonymity or censorship-resistance claims.

Table 6: What other parties can observe.

Party	Can observe
The other person	Your public IP address on a direct path, and your local addresses where the browser includes them as host candidates (in the lab, Chromium included 192.168.1.10; in the same-machine tests it used an mDNS name). Neither with relay-only mode.
The message channel	That a call is being set up and when; the SDP in the messages: candidate addresses, ICE credentials, certificate fingerprints, the session identifier, commitment and n_B. These values do not include media keys.
STUN servers External	Your public address and the time of each call set-up.
TURN relay External	Both participants' addresses, call timing and traffic volume; not the content.
Network providers	That encrypted WebRTC traffic flows between two addresses, with its timing and volume.

9 Build and testing

9.1 Test suites

The archive contains three automated suites. The *unit* suite (24 tests, Node.js test runner) covers the encoding layer, SDP inspection, schemas, the commitment and code, the gathering rule, candidate splitting and relay codes. It includes fuzzing: 1,500 envelope mutations, 5,000 structured-garbage objects across the schema parsers, and 4,000 SDP line mutations. The *end-to-end* suite (20 tests) runs separate Chromium processes on one machine with synthetic camera and microphone. It drives the real interface through Playwright, and a third browser page (`tests/e2e/attacker.html`) acts as a person in the middle. The *network lab* (12 tests) places each Chromium in its own Linux network namespace behind a simulated NAT router built with iptables. Both home networks use the same private range, and a real STUN/TURN server (pion/turn v4.1.4 [29]; TURN over UDP and TCP on port 3478 and TLS on 443) sits on a simulated public network. Routers drop unsolicited inbound traffic. The lab does not model packet loss, delay or jitter, and it does not model carrier-grade NAT.

9.2 What was run for this report

For this report the source archive was extracted to a separate directory and rebuilt; `node build.mjs --check` confirmed that the rebuilt file equals the delivered `PrivateCall.html` (SHA-256 `6310489f…268b`). All three suites were then run again from that copy in the following environment: a Linux container, Node.js 22, Playwright 1.63 with Chromium 153.0.8010.12, and, for the lab, root privileges with network namespaces and the relay server rebuilt from the archive. This rerun was carried out by the same party that developed PrivateCall, in the same kind of environment, not an independent reproduction. Table 7 gives the results.

Table 7: Results of the rerun for this report (October 2026, Chromium 153.0.8010.12, Linux).

Suite	Tests	Checks	Result and notable observations
Build check	–	1	Rebuilt file identical to the delivered one.
Unit and fuzz	24	–	24 passed, 0 failed.
End-to-end	20	129	20 passed, 129 checks, 0 failures. DTLS 1.3 with `SRTP_AES128_CM_HMAC_SHA1_80`; certificate check "matched"; invitation 1,872 and reply 1,840 characters; invitation ready after 15.1 s because public STUN is unreachable from the test machine; "Couldn't connect" shown 45.6 s after adding a reply from a closed window.
Network lab	12	38	12 passed, 38 checks, 0 failures. Direct path through two home NATs (invitation ready in 0.5 s); two symmetric NATs without relay: "Couldn't connect" after 45.7 s; relay over UDP, TCP and TLS/443 connected (TLS with certificate checks disabled; with an untrusted certificate Chromium refused the relay); relay-only invitation contained only the relay address; 8 s outage recovered, 40 s outage ended after 17 s; replies added after 90 s and 200 s connected.

9.3 What is not tested

No other browser engine was available: Firefox, Edge and Safari were not tested, so interoperability with them is *unverified*, even though the interface names Chrome, Edge and Firefox (Section 10.2). Also untested are real home and mobile networks, carrier-grade NAT, a relay with a publicly trusted TLS certificate (in the lab, TURN over TLS only connected with certificate checks disabled), behaviour under loss and delay, real microphones, cameras and headsets, screen readers, long calls, and the usability of the code comparison. Voice-imitation attacks were not tested.

10 Limitations, discrepancies and conclusion

10.1 Limitations

- **Security evidence.** The code protocol has not been formally analysed or independently reviewed. The implementation has not been audited.
- **Human factors.** Protection against an attacker in the middle depends on people comparing the code correctly; spoken comparison is vulnerable to voice imitation, and impersonation is not detected at all.
- **Connectivity.** Without a relay, calls between two symmetric NATs failed in the lab, and calls from networks that block UDP are expected to fail. No relay is provided, and running one requires a server and its operator's trust. Reachability of the public STUN servers may differ between networks and countries and was not tested.
- **Address exposure.** Direct calls reveal each side's public address to the other.
- **Manual exchange.** Two messages must be carried by hand; a long outage ends the call and needs a new exchange.
- **Media.** No lip synchronisation; quality, latency and echo handling on real hardware and networks are unmeasured.
- **Scope.** One-to-one only; English interface only.

10.2 Discrepancies between code, comments and documentation

The review found no discrepancy in which documentation claims a security property that the code lacks. The following differences were found; the code was treated as authoritative throughout.

1. The header comment of `session.js` (lines 4–6) lists `connecting` between `reply-ready` and `connected`. In the code only the starting side enters `connecting` (line 309); the joining side goes from `reply-ready` (line 382) directly to `connected` (line 446). Figure 7 follows the code.
2. The welcome text says "Calls go directly between two computers". With a relay, or when only a relayed path works, media pass through the relay. The *Privacy and safety* dialog states it accurately ("directly ... whenever possible").
3. The unsupported-browser message tells people to use "Chrome, Edge or Firefox". Only Chromium was tested; the project's own compatibility notes say so.
4. The local microphone level meter (`main.js` lines 314–319) keeps running for the whole session, including the call screen where it is not shown. Its output is used only on screen and is never transmitted. Its `aria-valuenow` attribute is never updated, so assistive technology reports it as 0.
5. The 15-minute limit on candidate re-supply only stops the re-supply and shows a notice. The joining side does not end the session itself; the person ends it.

10.3 Conclusion

PrivateCall is a small system whose security rests on two layers: standard browser encryption (DTLS-SRTP), and an application-level comparison of a six-digit code that binds both certificate fingerprints to values committed before the connection exists. Its own code is mainly concerned with making that comparison sound and hard to bypass: an unambiguous fingerprint, a commitment, a local-only confirmation

that gates outgoing media, and strict handling of all received input. The remaining work belongs to the browser, the network, and the people using it. The evidence consists of automated tests in Chromium on one Linux system and a simulated network. Behaviour in other browsers, on real networks, and with real people comparing codes has not been verified.

References

[1] J. Uberti, C. Jennings, E. Rescorla (Ed.). *JavaScript Session Establishment Protocol (JSEP)*. RFC 9429, IETF, 2024.

[2] A. Begen, P. Kyzivat, C. Perkins, M. Handley. *SDP: Session Description Protocol*. RFC 8866, IETF, 2021.

[3] A. Keränen, C. Holmberg, J. Rosenberg. *Interactive Connectivity Establishment (ICE)*. RFC 8445, IETF, 2018.

[4] E. Ivov, J. Uberti, P. Saint-Andre. *Trickle ICE: Incremental Provisioning of Candidates for the ICE Protocol*. RFC 8838, IETF, 2021.

[5] M. Petit-Huguenin et al. *Session Traversal Utilities for NAT (STUN)*. RFC 8489, IETF, 2020.

[6] T. Reddy et al. *Traversal Using Relays around NAT (TURN)*. RFC 8656, IETF, 2020.

[7] F. Audet, C. Jennings. *Network Address Translation (NAT) Behavioral Requirements for Unicast UDP*. RFC 4787, IETF, 2007.

[8] J. Fischl, H. Tschofenig, E. Rescorla. *Framework for Establishing a Secure Real-time Transport Protocol (SRTP) Security Context Using Datagram Transport Layer Security (DTLS)*. RFC 5763, IETF, 2010.

[9] D. McGrew, E. Rescorla. *DTLS Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP)*. RFC 5764, IETF, 2010.

[10] M. Baugher et al. *The Secure Real-time Transport Protocol (SRTP)*. RFC 3711, IETF, 2004.

[11] E. Rescorla, H. Tschofenig, N. Modadugu. *The Datagram Transport Layer Security (DTLS) Protocol Version 1.3*. RFC 9147, IETF, 2022.

[12] J. Lennox, C. Holmberg. *Connection-Oriented Media Transport over the Transport Layer Security (TLS) Protocol in the Session Description Protocol (SDP)*. RFC 8122, IETF, 2017.

[13] C. Holmberg, R. Shpount. *Session Description Protocol (SDP) Offer/Answer Considerations for Datagram Transport Layer Security (DTLS) and Transport Layer Security (TLS)*. RFC 8842, IETF, 2021.

[14] R. Jesup, S. Loreto, M. Tüxen. *WebRTC Data Channels*. RFC 8831, IETF, 2021.

[15] E. Rescorla. *WebRTC Security Architecture*. RFC 8827, IETF, 2021.

[16] E. Rescorla. *Security Considerations for WebRTC*. RFC 8826, IETF, 2021.

[17] P. Zimmermann, A. Johnston (Ed.), J. Callas. *ZRTP: Media Path Key Agreement for Unicast Secure RTP*. RFC 6189, IETF, 2011.

[18] JM. Valin, C. Bran. *WebRTC Audio Codec and Processing Requirements*. RFC 7874, IETF, 2016.

[19] A.B. Roach. *WebRTC Video Processing and Codec Requirements*. RFC 7742, IETF, 2016.

[20] P. Deutsch. *DEFLATE Compressed Data Format Specification version 1.3*. RFC 1951, IETF, 1996.

[21] S. Josefsson. *The Base16, Base32, and Base64 Data Encodings*. RFC 4648, IETF, 2006.

[22] W3C. *WebRTC: Real-Time Communication in Browsers*. W3C Recommendation.

[23] W3C. *Identifiers for WebRTC's Statistics API*. W3C Candidate Recommendation.

[24] W3C. *Content Security Policy Level 3*. W3C Working Draft.

[25] W3C. *Trusted Types*. W3C Working Draft.

[26] W3C. *Secure Contexts*. W3C Candidate Recommendation.

[27] M. Shirvanian, N. Saxena. Wiretapping via Mimicry: Short Voice Imitation Man-in-the-Middle Attacks on Crypto Phones. In *Proc. ACM CCS*, 2014.

[28] J. Uberti. *A REST API For Access To TURN Services*. Internet-Draft draft-uberti-behave-turn-rest-00, IETF, 2013 (expired).

[29] Pion. *pion/turn: a TURN server and client toolkit in Go*, version 4.1.4. `https://github.com/pion/turn`.

A Claims, source locations and tests

Test identifiers: U = unit test (`tests/unit/core.test.mjs` and `hardening.test.mjs`), E = end-to-end test (`tests/e2e/run.mjs`), N = network-lab test (`tests/netlab/run.mjs`). Names are abbreviated from the test titles.

Claim	Source	Tests
Code derivation and commitment as in Section 5.2	`sas.js`: `makeCommit`, `checkCommit`, `deriveSas`; `util.js`: `lengthPrefixed`	U “commitment binds the nonce and the session id”, “both sides derive the same code...”, “codes are roughly uniform”
Substituted certificates give different codes; nothing sent before confirmation	`session.js`: `_computeSas`, `audioAllowed`, `videoAllowed`, `_sync*Sender`	E “mitm: substituted certificates...”, E “video call: ...media gating”
Wrong, replayed or missing reveal ends the call	`session.js`: `_onControl`, `_onChannelOpen`	E “mitm: reveal replaced...”, “mitm: reveal withheld”, “mitm: forged commitment...”
Peer messages cannot verify or unlock media	`session.js`: `_onControl`, `confirmCode`; `protocol.js`: `parseControl`	E “mitm: injected control messages...”; U “control messages are validated”
Impersonation is not detected	Design limit	E “limit: impersonation...”
Exactly one fingerprint; strict SDP	`sdp.js`: `inspectSdp`	U “SDP inspection rejects...”, “real Chromium offer/answer...”, SDP fuzz; E “tamper: certificate fingerprint swapped...”
Exact schemas; tampered messages refused	`protocol.js`: `parseInvite`, `parseReply`	U “invite and reply schemas are strict”, schema fuzz; E “tamper: modified messages with valid checksums”
Envelope robustness and limits	`codec.js`: `encodeEnvelope`, `decodeEnvelope`; `util.js`: `inflateRawLimited`	U envelope tests, “inflate refuses output beyond the limit”, envelope fuzz; E “input errors...”
Replay refused	`session.js`: `usedInviteIds`, `acceptReply`	E “replay: invitations and replies work once”
Gathering policy	`gather.js`: `gatherDecision`; `session.js`: `_waitForGathering`	U “gathering: ...” (4 tests); N “UDP blocked, relay configured...”, “public STUN unreachable...”
Candidate re-supply for slow exchanges	`session.js`: `_refreshRemoteCandidates`; `sdp.js`: `splitCandidates`, `refreshFoundation`	U “splitCandidates...”, “refreshFoundation...”; N “slow exchange ... 90 s / 200 s”; E “slow exchange: ... 60 s”
Direct path through home NATs	`ice.js`: `buildIceConfig`	N “home NATs: direct connection through STUN”
Relay over UDP, TCP and TLS; relay-only mode	`ice.js`: `buildIceConfig`, `parseRelayCode`	N symmetric/UDP-blocked/TCP-443 relay tests, “relay-only mode hides both addresses”; U “relay codes are validated and expire”, “relay-only mode does not contact public STUN”
Clear failure without a usable path	`session.js`: `acceptReply` timer; `main.js`: `diagnosticsText`	N “symmetric NATs, no relay: fails clearly”; E “failure: reply from a closed window...”
Recovery and loss of connection	`session.js`: `_onConnectionState`	N “connection blip of 8 s recovers; outage of 40 s ends the call”; E “remote reload...”
No network requests from the page; CSP	`build.mjs`: CSP; `index.html`	Every E test (fails on HTTP request or CSP violation)
Interface flows	`main.js`	E “files...”, “clipboard...”, “verification UI...”, “settings...”, “layout: right-to-left...”, “voice call...”